STUDENT PERFORMANCE ANALYTICS SYSTEM

1. Project Title

Student Performance Analytics System

Course: Python with AI | Student: Vivek Kumar

Repository: vivekkr-data/student-performance-analytics. Assignment deadline: 15 October 2026.

The application was executed on the supplied Kaggle dataset. All 30 records passed validation, analysis completed successfully, and 31 automated software tests passed.

2. Objective

The objective is to turn a student CSV into understandable academic summaries. The system calculates totals, averages, grades and pass/fail status, then compares student, subject and department performance. Attendance is reported separately.

The project uses the Python concepts taught in the course and keeps the calculations separate from display and file export so that each part can be explained and tested.

3. Technologies Used

Python: program flow, functions, loops, conditions, command-line options, validation and standard unittest tests.

NumPy: arrays, row sums and means, class median, population standard deviation and correlation.

Pandas: CSV loading, DataFrames, inspection, filtering, sorting, aggregation, groupby and report exports.

CSV stores the student input and analysis exports. GitHub hosts the project repository. Tested environment: Python 3.12.14, NumPy 2.3.5 and Pandas 2.2.3. No machine-learning libraries are required.

DATA & IMPLEMENTATION

4. Dataset Description

Kaggle source: TSS Pandas Challenge #1, Gagan H (2026). Kaggle dataset page

The supplied archive contains student_data.csv. It was copied without byte changes to students.csv. The actual dataset contains 30 records and 9 columns: student_id, name, department, year, age, attendance, python_marks, ml_marks and projects_completed.

The data represents a Pandas practice dataset. All 9 columns have zero missing values; all student IDs are unique. Marks and attendance are within 0-100. No rows were removed or fabricated. Year, age and completed projects are retained but excluded from exam marks.

The source CSV has no explicit maximum-score field. Both subjects are treated as out of 100 under the declared project model. The archive has no license file; this project supplies attribution and does not assert an independently verified open-data license. Full provenance and checksum appear in DATASET_SOURCE.md.

5. Implementation

1. load_student_data reads the CSV with Pandas and preserves student IDs as text. Duplicate CSV headers are rejected before Pandas can rename them.

2. validate_student_data checks required fields, record count, duplicate IDs and numeric ranges. It returns a copy and does not modify the source file.

3. calculate_student_results uses a NumPy matrix. For m equally weighted subjects out of 100: total = sum of scores; average = total / m; percentage = total / (100 x m) x 100.

4. determine_pass_status requires at least 40 in every subject. assign_grade uses a loop and conditions; any subject failure gives overall FAIL and F. Otherwise percentage thresholds 90, 80, 70, 60, 50 and 40 give A+, A, B, C, D and E. These are project assumptions, not institutional rules.

5. Class, subject, department and attendance functions return calculated summaries. Standard deviation is population-based (ddof=0). Ranking uses unrounded scores; displays use two decimals.

6. main.py builds the report, exports CSVs and text, then displays the analysis. Missing inputs and ordinary filesystem errors produce a readable error and a nonzero exit code.

FEATURES & OUTPUT

6. Key Features

Dataset overview: first five records, dimensions, column names, data types, missing counts and descriptive statistics.

Student and class performance: totals, averages, percentages, grades, pass/fail counts, grade distribution, mean, median and standard deviation. All tied highest and lowest students are included.

Subject performance: mean, highest, lowest, median, standard deviation, passing counts and percentages. Strongest and weakest subjects are identified by their means.

Top performers: five rows sorted by descending average and total, then ascending text student ID. Equal scores share rank. At the fifth-row boundary, ID order selects which tied records appear.

Department comparison: count, average and pass rate using groupby. A single department is explicitly identified as unsuitable for cross-department comparison.

Attendance: mean, maximum, minimum and students below 75%. Pearson correlation is omitted as undefined when either variable is constant. Attendance does not change exam pass/fail status.

The verified run produced student_results.csv, subject_analysis.csv, department_analysis.csv and performance_summary.txt. Original student columns are retained alongside computed results.

7. Output / Screenshots (Optional)

The report uses genuine output from the executed application. The complete console output is preserved in documentation/evidence/actual_run.txt; test output is in test_run.txt. No screenshot images are fabricated.

Actual summary excerpt: 30 passed and 0 failed out of 30 students. Class average: 82.35/100. Highest performer: Sanya (22). Lowest performer: Manish (21). Strongest subject by mean: ml_marks.

The application produced all required sections and exited with code 0. Actual text evidence is included because screenshot images are optional in the assignment.

RESULTS & SUBMISSION

8. Final Outcome

The system analyzed all 30 supplied records successfully. Under the rule of at least 40 in each subject, every student passed: 30 passed, 0 failed, pass percentage 100.00%.

The class average was 82.35/100. Sanya (ID 22) ranked highest at 97.50/100 (195/200); Manish (ID 21) ranked lowest at 63.00/100 (126/200).

Python averaged 81.40/100 and ML averaged 83.30/100. ML was stronger by 1.90 marks. AIML had the highest department mean at 92.06; CSE had the lowest at 77.30.

Average attendance was 82.60%; 7 students (23.33%) had attendance below 75%. Low attendance was reported separately and did not change exam status.

All 31 automated tests passed, including synthetic failure cases and grade boundaries. Separately, standard-library csv and statistics calculations independently checked all student totals and averages and the class summary against the exported results.

9. Challenges & Learning

The initial automated Kaggle download required authentication. This was resolved when the user supplied the original downloaded archive. The actual member filename used an underscore rather than the hyphen described on the source page; the file was copied without modifying its contents.

Boundary handling, failure in one subject despite a high average, tied ranks and undefined correlation were addressed with explicit conditions and executable tests. No errors or missing values required cleaning in the supplied dataset.

The work demonstrates NumPy array calculations, Pandas grouping and filtering, reusable functions, input validation, independent verification and repository organization. The student should understand and be able to explain these functions during evaluation.

10. GitHub Repository

GitHub repository: Open GitHub repository

The repository contains Python source, the supplied CSV, dependencies, README, tests, generated reports and this documentation. The target repository was initially empty; no unrelated project files were overwritten.

Paste this repository URL into the EWB Google Form and submit before the deadline. The form URL was not supplied and form submission has not been performed.